

# Juego del Ahorcado en Python

## Documento Técnico del Sistema

Funcionalidades | Estructura | Organización | Herramientas | Impacto Social

|            |                      |
|------------|----------------------|
| Autor      | Santiago López       |
| Asignatura | Programación         |
| Fecha      | 27 de junio del 2026 |
| Lenguaje   | Python 3.x           |

# 1. Introducción

---

La programación es una de las competencias más relevantes del siglo XXI. En el contexto educativo ecuatoriano, la asignatura de Informática y Programación del Bachillerato General Unificado (BGU) tiene como propósito fundamental introducir a los estudiantes en el pensamiento computacional, el desarrollo de algoritmos y la construcción de soluciones tecnológicas que respondan a necesidades concretas.

El presente documento describe el desarrollo del Juego del Ahorcado implementado en lenguaje Python, un proyecto de programación educativa diseñado y ejecutado en el marco de ayudar al razonamiento lúdico y a mejorar la atención en los usuarios. Este programa integra de forma práctica los principales contenidos del currículo de programación: estructuras de control, funciones, estructuras de datos, algoritmos y lógica de programación.

El Ahorcado es un juego clásico de adivinanza de palabras ampliamente conocido en el mundo hispanohablante. Su adaptación como programa de computadora ofrece un contexto motivador y familiar para que los usuarios experimenten de primera mano cómo los conceptos abstractos de la programación se convierten en soluciones reales, funcionales e interactivas.

El sistema permite al jugador ingresar su nombre, seleccionar una categoría temática de entre siete opciones disponibles (Animales, Cosas, Frutas, Países, Deportes, Nombres de personas y Programación), adivinar una palabra letra por letra con un máximo de seis errores permitidos, y recibir mensajes personalizados al finalizar cada partida. Todo el programa está construido únicamente con elementos nativos de Python, sin dependencias externas, lo que lo hace accesible y portable en cualquier entorno educativo.

i Este proyecto fue desarrollado como material didáctico. Puede ser analizado, modificado y ampliado libremente por los estudiantes como parte de su proceso de aprendizaje.

## 2. Descripción del Problema

### 2.1 Contexto y Situación Inicial

La utilidad práctica de los conceptos teóricos que aprenden en clase. Conceptos como las estructuras de control, las funciones, los bucles y las estructuras de datos suelen percibirse como abstractos y desconectados de aplicaciones reales cuando se presentan de forma aislada.

Adicionalmente, en muchos contextos educativos los recursos tecnológicos son limitados: se trabaja con computadoras de baja capacidad, sin acceso constante a internet y con restricciones para instalar software adicional. Esto limita el tipo de proyectos que pueden desarrollarse en el aula.

### 2.2 Problema Identificado

Se identifico la necesidad de contar con un proyecto de programación que cumpliera simultáneamente con los siguientes criterios:

- **Pedagógico:** que integrara de forma natural los contenidos del currículo de programación.
- **Motivador:** que generara interés genuino en los usuarios mediante un formato lúdico y familiar.
- **Accesible:** que funcionara en cualquier computadora con Python instalado, sin requerir internet ni librerías externas.
- **Escalable:** que pudiera ampliarse con nuevas funcionalidades a medida que los estudiantes avanzan en su desarrollo.
- **Evaluable:** que permitiera al docente verificar la comprensión de conceptos específicos de programación a través del análisis del código.

### 2.3 Solución Propuesta

Como respuesta a esta necesidad, se diseño e implemento el Juego del Ahorcado en Python: una aplicación de consola que recrea el clásico juego de adivinanza de palabras, incorporando elementos de personalización (nombre del jugador), organización temática (categorías de palabras), retroalimentación visual (arte ASCII) y experiencia de usuario (mensajes personalizados, validación de entradas, opción de reinicio).

| Necesidad identificada             | Solución implementada en el programa                               |
|------------------------------------|--------------------------------------------------------------------|
| Proyecto motivador y concreto      | Juego interactivo con nombre del jugador y categorías temáticas.   |
| Sin dependencias externas          | Solo usa el módulo random de la librería estándar de Python.       |
| Contenidos curriculares integrados | Usa funciones, bucles, condicionales, sets, listas y diccionarios. |
| Retroalimentación visual           | Arte ASCII de la horca con 7 etapas progresivas.                   |
| Experiencia personalizada          | Mensajes con nombre del jugador en victoria, derrota y despedida.  |

| Necesidad identificada | Solución implementada en el programa                                    |
|------------------------|-------------------------------------------------------------------------|
| Escalabilidad futura   | Estructura modular que facilita agregar categorías, niveles o interfaz. |

## 3. Objetivo del Sistema

---

### 3.1 Objetivo General

Desarrollar un programa interactivo de consola en Python que implemente el Juego del Ahorcado con selección de categorías y personalización del jugador, integrando los contenidos fundamentales de la asignatura de Programación, con el fin de fortalecer el pensamiento computacional y las habilidades de programación de los estudiantes.

### 3.2 Objetivos Específicos

Los siguientes objetivos específicos orientan el desarrollo y la evaluación del sistema:

1. **Implementar una interfaz de consola interactiva** que permita al jugador ingresar su nombre, seleccionar una categoría y participar en el juego de forma intuitiva.
2. **Integrar siete categorías temáticas** con 25 palabras cada una, seleccionadas aleatoriamente mediante la función `random.choice()`.
3. **Aplicar estructuras de control** (`while`, `if/else`, `break`, `continue`) para gestionar el flujo del juego y detectar las condiciones de victoria o derrota.
4. **Utilizar estructuras de datos adecuadas** (listas, diccionarios, conjuntos y cadenas) para organizar las palabras y registrar las letras ingresadas.
5. **Desarrollar funciones modulares** con responsabilidad única que separen claramente las responsabilidades del programa.
6. **Proporcionar retroalimentación visual** mediante el arte ASCII de la horca y mensajes personalizados con el nombre del jugador.
7. **Validar todas las entradas del usuario** garantizando robustez del programa ante entradas incorrectas, repetidas o vacías.
8. **Demostrar la aplicación practica** de los conceptos de programación del currículo en un proyecto funcional y documentado.

### 3.3 Alcance del Sistema

- **Incluye:** interfaz de consola, 7 categorías con 25 palabras, personalización, validación, arte ASCII, mensajes de resultado y opción de reinicio.
- **No incluye:** interfaz gráfica, base de datos, multijugador en red ni sistema de puntuación persistente.
- **Plataforma:** multiplataforma (Windows, macOS, Linux). Requiere Python 3.6 o superior.
- **Público objetivo:** usuarios desde los siete años en adelante.

## 4. Relación con los Contenidos de la Asignatura de Programación

El Juego del Ahorcado integra de manera orgánica y practica los principales contenidos del currículo de la asignatura de Programación. A continuación se explica en detalle como cada contenido curricular se manifiesta en el programa.

### 4.1 Entorno de Programación

El entorno de programación es el conjunto de herramientas, plataformas y configuraciones que permiten escribir, ejecutar y depurar código. Este proyecto utiliza:

- **Python 3:** lenguaje de alto nivel, interpretado, de código abierto y multiplataforma. Su sintaxis clara lo convierte en el lenguaje ideal para la enseñanza de la programación.
- **Interprete de Python:** el programa se ejecuta desde la terminal del sistema operativo con el comando `Python ahorcado.py`.
- **Editor de texto o IDE:** el código puede editarse con VS Code, PyCharm, Thonny, IDLE o cualquier editor de texto, eliminando barreras de acceso.
- **Librería estándar:** se usa el módulo `random`, parte de la instalación básica de Python. No se requiere ninguna instalación adicional.

#### Terminal de comandos

```
# Verificar la version de Python instalada
python --version

# Ejecutar el programa desde la terminal
python ahorcado.py
```

i El uso de la terminal es en si mismo un aprendizaje fundamental: los estudiantes desarrollan habilidades de navegación por el sistema de archivos y ejecución de programas desde la línea de comandos.

### 4.2 Manejo de Datos

El manejo de datos abarca la forma en que el programa almacena, organiza, accede y modifica la información durante su ejecución. En el Ahorcado se trabajan los siguientes tipos de datos:

| Tipo de dato    | Variable / Uso           | Ejemplo en el programa                                         |
|-----------------|--------------------------|----------------------------------------------------------------|
| str (cadena)    | nombre, palabra, letra   | <code>nombre = "Edison"</code> <code>letra.lower()</code>      |
| int (entero)    | errores, max_errores     | <code>errores = len(letras_incorrectas)</code>                 |
| bool (booleano) | Resultado de condiciones | <code>letra.isalpha()</code> <code>opcion in CATEGORIAS</code> |
| list (lista)    | HORCA, lista de palabras | <code>HORCA[errores]</code> <code>categoria['palabras']</code> |

| Tipo de dato       | Variable / Uso                           | Ejemplo en el programa     |
|--------------------|------------------------------------------|----------------------------|
| dict (diccionario) | CATEGORIAS                               | CATEGORIAS["1"]["nombre"]  |
| set (conjunto)     | letras_adivinadas,<br>letras_incorrectas | letras_adivinadas.add('a') |

El manejo de datos también incluye métodos de cadena: `.lower()` para normalizar, `.strip()` para eliminar espacios, `.isalpha()` para verificar letra y `.upper()` para el resultado final.

### 4.3 Algoritmos

Un algoritmo es una secuencia finita y ordenada de pasos que resuelve un problema específico. El programa implementa cuatro algoritmos claramente definidos:

#### Algoritmo 1: Selección de palabra aleatoria

9. Recibir el diccionario de la categoría seleccionada.
10. Acceder a la lista de palabras: `categoría['palabras']`.
11. Aplicar `random.choice()` para seleccionar un elemento con distribución uniforme.
12. Retornar la palabra seleccionada para iniciar la partida.

#### Algoritmo 2: Visualización de la palabra

13. Iterar sobre cada carácter 'c' de la cadena 'palabra'.
14. Si 'c' está en letras adivinadas, mostrar 'c'; si no, mostrar '\_ '.
15. Unir los resultados con espacios: `' '.join(...)`.
16. Imprimir el resultado junto a la cantidad de letras.

#### Algoritmo 3: Verificación de fin de partida

17. Calcular errores = `len(letras_incorrectas)`.
18. Verificar victoria: `all(c in letras_adivinadas for c in palabra)`.
19. Verificar derrota: `errores >= max_errores (6)`.
20. Si ninguna condición se cumple, continuar el bucle de juego.

#### Algoritmo 4: Validación de letra

21. Leer entrada del usuario con `input()` y normalizar con `.lower().strip()`.
22. Verificar longitud == 1 y carácter alfabético con `.isalpha()`.
23. Verificar que no haya sido ingresada antes.
24. Clasificar: si está en 'palabra' agregar a adivinadas; si no, a incorrectas.

### 4.4 Flujogramas

Los flujogramas son representaciones gráficas de los algoritmos y el flujo de control de un programa. Para este sistema se desarrollaron 11 flujogramas completos en un documento PDF separado:

- **Modulo 1:** Punto de entrada (`__main__`): flujo de registro del jugador y validación del nombre.

- **Modulo 2:** elegir\_categoria(): bucle del menu y validacion de opcion 1-7.
- **Modulo 3 Parte 1:** ahorcado(): inicializacion y bucle principal del juego.
- **Modulo 3 Parte 2:** ahorcado(): fin de partida y opcion de reinicio.
- **Modulo 4:** Validación y procesamiento de la letra ingresada.
- **7 flujogramas de categorías:** uno por cada categoría con flujo de selección aleatoria y banco de 25 palabras.

! Los flujogramas utilizan la simbología estándar ISO: óvalos (inicio/fin), rectángulos (procesos), rombos (decisiones) y flechas (flujo). El documento PDF de flujogramas es un entregable complementario a este informe.

## 4.5 Lógica de Programación

La lógica de programación es la capacidad de organizar instrucciones de forma coherente para resolver un problema. En el Ahorcado se aplican los tres principios fundamentales:

| Principio   | Descripción                  | Aplicación en el Ahorcado                                                  |
|-------------|------------------------------|----------------------------------------------------------------------------|
| Secuencia   | Instrucciones en orden       | Mostrar estado -> verificar -> pedir letra -> registrar -> repetir.        |
| selección   | Tomar decisiones (if/else)   | Si letra correcta: adivinadas. Si errores=6: derrota. Si all(): victoria.  |
| Repetición  | Repetir bloques (while)      | Repetir el juego hasta ganar/perder. Repetir menu hasta opcion valida.     |
| Modularidad | Dividir en funciones         | mostrar_menu(), elegir_categoria() y ahorcado() con responsabilidad unica. |
| Abstracción | Representar conceptos reales | Una letra -> str. Un intento fallido -> elemento en set. Errores -> int.   |
| Recursión   | Función que se autoinvoca    | ahorcado(nombre) se llama a si misma para iniciar una nueva partida.       |

## 4.6 Bucles

Los bucles son estructuras que permiten repetir un bloque de instrucciones multiples veces. El programa implementa tres bucles while con propósitos específicos:

### Bucle 1: Validación del nombre (while not nombre)

#### Bucle de validación de nombre

```
nombre = input('Ingresa tu nombre: ').strip()
while not nombre:      # True mientras nombre este vacio
    nombre = input('El nombre no puede estar vacio: ').strip()
```

Garantiza que el programa no avance hasta recibir un nombre no vacío. 'not nombre' es True cuando la cadena esta vacía.



## Bucle 2: Menú de categorías (while True con return)

### Bucle del menu

```
def elegir_categoria():
    while True:
        mostrar_menu_categorias()
        opcion = input('Categoria: ').strip()
        if opcion in CATEGORIAS:
            return CATEGORIAS[opcion] # Sale del bucle
        else:
            print('Opcion no valida.')
```

El while True se termina con return cuando el usuario ingresa una opcion valida. De lo contrario el menu se repite indefinidamente.

## Bucle 3: Juego principal (while True con break)

### Bucle principal del juego

```
while True:
    errores = len(letras_incorrectas)
    print(HORCA[errores])
    if all(c in letras_adinadas for c in palabra):
        print(f'Ganaste, {nombre}!') # Victoria
        break # Sale del bucle
    if errores >= max_errores:
        print(f'Lo siento, {nombre}!') # Derrota
        break # Sale del bucle
    letra = input('Letra: ').lower().strip()
    if len(letra) != 1 or not letra.isalpha():
        continue # Vuelve al inicio sin contar
    # ... registrar letra
```

Es el corazón del programa. Se ejecuta hasta que break lo detenga. La instrucción continue salta al inicio del bucle cuando la entrada no es válida, sin penalizar al jugador.

## 4.7 Estructuras de Datos

Las estructuras de datos son formas organizadas de almacenar informacion. El Ahorcado usa cuatro estructuras nativas de Python:

### Lista — HORCA y listas de palabras

#### Lista

```
HORCA = [ '"" +---+ ... ""', '"" +---+ O... ""', ... ]
print(HORCA[errores]) # Acceso O(1) por indice
random.choice(categoria['palabras']) # Elemento aleatorio de lista
```

Mantiene orden y permite acceso por indice. El indice de HORCA coincide exactamente con el numero de errores, haciendo el acceso directo y eficiente.

### Diccionario — CATEGORIAS

#### Diccionario

```

CATEGORIAS = {
    "1": { "nombre": "Animales", "palabras": ["elefante", ...] },
    "2": { "nombre": "Cosas", "palabras": ["silla", ...] }, ... }

categoria = CATEGORIAS["1"]          # Acceso O(1) por clave
opcion in CATEGORIAS                 # Verificacion O(1)

```

Asocia claves con valores. Permite validar la opción del menú y acceder a los datos de la categoría con acceso en tiempo constante.

## Conjunto — letras\_adivinadas y letras\_incorrectas

### Conjunto (set)

```

letras_adivinadas = set()           # Sin duplicados
letras_adivinadas.add('a')          # Agregar: O(1)
'a' in letras_adivinadas            # Buscar: O(1)
errores = len(letras_incorrectas)   # Contar errores

```

Ideal porque no permite duplicados (una letra no se registra dos veces) y la búsqueda es  $O(1)$ , mas eficiente que en una lista.

## Cadena — palabra, nombre, letra

### Cadenas de texto

```

letra.lower()      # Normalizar: 'A' -> 'a'
letra.strip()      # Limpiar: ' a ' -> 'a'
letra.isalpha()    # Validar: True si es letra
nombre.upper()     # Resultado: 'edison' -> 'EDISON'
for c in palabra:  # Iterar caracter por caracter

```

Las cadenas son iterables en Python, lo que permite construir la representación visual de la palabra carácter por carácter. Sus métodos son esenciales para validar y transformar el texto ingresado por el usuario.

## 5. Funcionalidades del Sistema

El Juego del Ahorcado desarrollado en Python es una aplicación de consola interactiva que combina entretenimiento con aprendizaje. A continuación, se detallan todas sus funcionalidades organizadas por categoría.

### 5.1 Registro del Jugador

Al iniciar el programa, el sistema solicita al usuario que ingrese su nombre antes de comenzar cualquier partida. Esta funcionalidad personaliza completamente la experiencia del jugador.

- Solicitud del nombre mediante la función `input()` de Python.
- Validación que impide continuar si el campo está vacío.
- Uso del nombre en todos los mensajes del juego: bienvenida, pistas, victoria y derrota.
- El nombre se conserva durante toda la sesión, incluso al reiniciar partidas.

i El nombre del jugador se convierte a mayúsculas en los mensajes finales usando el método `.upper()`, lo que da mayor impacto visual al mensaje de victoria o derrota.

### 5.2 Menú de Categorías

Una vez registrado, el jugador accede a un menú interactivo que le permite escoger la categoría temática de la palabra que deberá adivinar. El sistema ofrece siete categorías con 25 palabras cada una.

| # | Categoría           | Descripción                                                    | Palabras |
|---|---------------------|----------------------------------------------------------------|----------|
| 1 | Animales            | Fauna del planeta: mamíferos, reptiles, aves, peces e insectos | 25       |
| 2 | Cosas               | Objetos del hogar, escuela y vida cotidiana                    | 25       |
| 3 | Frutas              | Frutas tropicales, templadas y exóticas del mundo              | 25       |
| 4 | Países              | Naciones de América Latina, Europa y Asia                      | 25       |
| 5 | Deportes            | Disciplinas deportivas olímpicas y no olímpicas                | 25       |
| 6 | Nombres de personas | Nombres propios masculinos y femeninos en español              | 25       |
| 7 | Programación        | Términos técnicos del mundo de la programación y computación   | 25       |

### 5.3 Mecánica de Juego

El núcleo del juego implementa la lógica clásica del ahorcado con mejoras de experiencia de usuario. El jugador dispone de 6 intentos fallidos antes de perder la partida.

- Selección aleatoria de una palabra dentro de la categoría elegida usando `random.choice()`.

- Visualización de la palabra como guiones bajos ( \_ ) que se revelan al acertar una letra.
- Representación grafica de la horca en arte ASCII con 7 etapas de construcción (0 a 6 errores).
- Contador de errores visible en cada turno con formato errores/máximo.
- Listado de letras incorrectas ya ingresadas para ayudar al jugador.
- Indicador del número total de letras de la palabra como pista adicional.
- Información de la categoría activa visible durante toda la partida.

## 5.4 Validación de Entradas

El programa incluye un sistema robusto de validación que garantiza entradas correctas en todo momento, evitando errores y trampas.

- **Validación de nombre:** no acepta campos vacíos; repite la solicitud hasta recibir un nombre valido.
- **Validación de opción de categoría:** solo acepta números del 1 al 7; muestra error si el valor es incorrecto.
- **Validación de letra:** verifica que la entrada sea exactamente un carácter alfabético.
- **Control de duplicados:** detecta letras ya ingresadas (correctas o incorrectas) e informa al jugador sin penalizarlo.

## 5.5 Mensajes Personalizados de Resultado

Al finalizar cada partida, el sistema muestra un mensaje destacado con el nombre del jugador según el resultado obtenido:

- **Victoria:** despliega el mensaje 'FELICITACIONES, [NOMBRE]!' junto con la palabra adivinada y la categoría.
- **Derrota:** muestra 'LO SIENTO, [NOMBRE]!' revelando la palabra secreta que no fue descubierta.

## 5.6 Opción de Repetición

Al terminar cada partida, el sistema pregunta si el jugador desea jugar nuevamente. Si elige que si, puede seleccionar una nueva categoría y comenzar una partida fresca manteniendo su nombre registrado. Si decide salir, el programa muestra un mensaje de despedida personalizado.

## 6. Estructura Lógica del Programa

---

La lógica del programa sigue un flujo secuencial bien definido, con estructuras de control que gestionan el flujo de ejecución en cada etapa del juego. El programa se divide en cuatro grandes bloques lógicos.

### 6.1 Bloque de Inicialización

Este bloque se ejecuta una sola vez al iniciar el programa. Su responsabilidad es preparar el entorno y registrar al jugador.

1. El intérprete de Python carga el módulo random al inicio del archivo.
2. Se definen todas las estructuras de datos: el diccionario CATEGORIAS con 7 entradas de 25 palabras cada una, y la lista HORCA con 7 dibujos ASCII.
3. El bloque `if __name__ == '__main__'` detecta que el archivo se ejecuta directamente.
4. Se muestra el banner del juego y se solicita el nombre del jugador.
5. Se valida que el nombre no este vacío con un bucle while.

### 6.2 Bloque de Selección de Categoría

Una vez registrado el jugador, el programa ejecuta la función `elegir categoría()` que gestiona el menú interactivo de seleccion.

- Se invoca `mostrar_menu_categorias()` que imprime el listado numerado de opciones.
- El programa lee la opción del usuario y verifica si existe como clave en el diccionario CATEGORIAS.
- Si la opción no es válida, muestra un mensaje de error y repite el proceso en un bucle `while True`.
- Si es válida, retorna el diccionario correspondiente a la categoría seleccionada.

### 6.3 Bloque del Bucle de Juego

Es el núcleo del programa. Implementado como un bucle `while True` que se ejecuta repetidamente hasta que ocurra una condición de fin. En cada iteración se realizan las siguientes operaciones en orden:

6. Calcular el número actual de errores: `len(letras_incorrectas)`.
7. Imprimir el dibujo de la horca correspondiente: `HORCA[errores]`.
8. Construir y mostrar la palabra con letras reveladas y guiones.
9. Mostrar categorial, letras incorrectas y contador de errores.
10. Verificar condición de victoria: `all(c in letras_adinadas for c in palabra)`.
11. Verificar condición de derrota: `errores >= max_errores (6)`.
12. Si hay fin, mostrar mensaje personalizado y ejecutar `break`.
13. Si no hay fin, solicitar una letra al jugador.
14. Validar la entrada (longitud, tipo, duplicado).
15. Clasificar la letra como correcta o incorrecta y agregarla al conjunto correspondiente.

## 6.4 Bloque de Fin de Partida

Tras salir del bucle principal, el programa ejecuta la logica de cierre de la partida.

- Pregunta al jugador si desea jugar otra vez con `input('¿Quieres jugar otra vez? (s/n)')`.
- Si el jugador responde 's', se hace una llamada recursiva a `ahorcado(nombre)`, reiniciando el juego con el mismo nombre, pero pidiendo una nueva categoría.
- Si responde cualquier otra tecla, muestra la despedida personalizada y el programa termina.

## 6.5 Diagrama de la Lógica de Control

La siguiente tabla resume las estructuras de control utilizadas y su propósito dentro del programa:

| Estructura                                | Ubicación en el código          | Propósito lógico                             |
|-------------------------------------------|---------------------------------|----------------------------------------------|
| <code>while not nombre</code>             | Punto de entrada                | Forzar ingreso de nombre valido.             |
| <code>while True (menu)</code>            | <code>elegir_categoria()</code> | Repetir menú hasta opción valida.            |
| <code>while True (juego)</code>           | <code>ahorcado(nombre)</code>   | Bucle principal de la partida.               |
| <code>if all(...)</code>                  | Dentro del bucle                | Detectar victoria del jugador.               |
| <code>if errores &gt;= max_errores</code> | Dentro del bucle                | Detectar derrota del jugador.                |
| <code>if not letra.isalpha()</code>       | Validación                      | Rechazar entradas no alfabéticas.            |
| <code>if letra in letras_...</code>       | Validación                      | Detectar letras repetidas.                   |
| <code>if letra in palabra</code>          | Procesamiento                   | Clasificar letra como correcta o incorrecta. |
| <code>if otra == 's'</code>               | Fin de partida                  | Decidir reinicio o salida.                   |

## 7. Organización del Código

El programa sigue los principios de programación estructurada y modular. El código está organizado en secciones claramente diferenciadas que facilitan su lectura, mantenimiento y extensión futura.

### 7.1 Estructura General del Archivo

El archivo ahorcado.py se organiza de arriba hacia abajo en el siguiente orden:

| Seccion               | Lineas aprox. | Contenido                                            |
|-----------------------|---------------|------------------------------------------------------|
| Importaciones         | 1             | import random — única dependencia externa.           |
| Datos: CATEGORIAS     | 3 - 75        | Diccionario con 7 categorías y 25 palabras cada una. |
| Datos: HORCA          | 77 - 134      | Lista de 7 dibujos ASCII del ahorcado.               |
| Función: mostrar_menu | 137 - 143     | Imprime el menú numerado de categorías.              |
| Función: elegir_cat.  | 146 - 155     | Valida y retorna la categoría seleccionada.          |
| Función: ahorcado()   | 158 - 222     | Lógica completa de la partida.                       |
| Punto de entrada      | 225 - 233     | Bloque if __name__ == '__main__'.                    |

### 7.2 Modulo 1: Datos Globales

Las constantes de datos se definen en el nivel global del módulo, fuera de cualquier función. Esto permite que sean accesibles desde cualquier parte del programa sin necesidad de pasarlas como parámetros.

#### Seccion de datos globales

```
# Diccionario de categorias
CATEGORIAS = {
    "1": { "nombre": "Animales", "palabras": ["elefante", "tigre", ...] },
    "2": { "nombre": "Cosas",      "palabras": ["silla", "mochila", ...] },
    ... # 7 categorias en total
}

# Lista de dibujos ASCII (7 etapas)
HORCA = [ """...""", """...""", ... ] # indices 0 a 6
```

! El uso de nombres en MAYUSCULAS para CATEGORIAS y HORCA es una convención de Python (PEP 8) que indica que son constantes: valores que no deben modificarse durante la ejecución.

### 7.3 Modulo 2: mostrar\_menu\_categorias()

Función auxiliar de responsabilidad única: únicamente imprime el menú. No recibe parámetros ni retorna valores. Esta separación permite reutilizarla fácilmente desde otras partes del código.

```
def mostrar_menu_categorias():  
  
def mostrar_menu_categorias():  
    print("\n" + "=" * 40)  
    print("      ELIGE UNA CATEGORIA")  
    print("=" * 40)  
    for clave, info in CATEGORIAS.items():  
        print(f"    [{clave}] {info['nombre']}")  
    print("=" * 40)
```

### 7.4 Modulo 3: elegir\_categoria()

Función que gestiona la interacción del menú. Implementa un bucle while True que garantiza que el programa no avance hasta recibir una opción válida. Retorna el diccionario completo de la categoría elegida.

```
def elegir_categoria():  
  
def elegir_categoria():  
    while True:  
        mostrar_menu_categorias()  
        opcion = input("Ingresa el numero de categoria: ").strip()  
        if opcion in CATEGORIAS:  
            print(f"Categoria: {CATEGORIAS[opcion]['nombre']}")  
            return CATEGORIAS[opcion]  
        else:  
            print("Opcion no valida. Elige del 1 al 7.")
```

### 7.5 Modulo 4: ahorcado(nombre)

Es la función principal del juego. Recibe el nombre del jugador como parámetro y encapsula toda la lógica de una partida: selección de palabra, bucle de juego, validaciones, verificación de resultados y opción de repetición.

```
def ahorcado(nombre) — resumen  
  
def ahorcado(nombre):  
    categoria = elegir_categoria()  
    palabra = random.choice(categoria['palabras'])  
    letras_adinadas = set()  
    letras_incorrectas = set()  
    max_erros = 6  
    while True:  
        errores = len(letras_incorrectas)  
        print(HORCA[errores]) # Dibujo actual  
        display = ' '.join(c if c in letras_adinadas else '_' for c in palabra)  
        print(f'Palabra: {display}')  
        if all(c in letras_adinadas for c in palabra): break # Victoria  
        if errores >= max_erros: break # Derrota  
        letra = input('Ingresa una letra: ').lower().strip()  
        # ... validaciones y procesamiento
```



```
otra = input('¿Jugar otra vez? (s/n): ')
if otra == 's': ahorcado(nombre)      # Recursion
```

## 7.6 Punto de Entrada

El bloque `if __name__ == '__main__':` es el punto de entrada del programa. Se ejecuta únicamente cuando el archivo se corre directamente desde la terminal, no cuando es importado como módulo por otro script.

### Punto de entrada `__main__`

```
if __name__ == '__main__':
    print("=====")
    print("      JUEGO DEL AHORCADO")
    print("=====")
    nombre = input("Ingresa tu nombre: ").strip()
    while not nombre:
        nombre = input("El nombre no puede estar vacío: ").strip()
    print(f"Hola, {nombre}! Bienvenido.")
    ahorcado(nombre)
```

## 8. Uso de Herramientas de Desarrollo

El desarrollo del Juego del Ahorcado hace uso de diversas herramientas, conceptos y elementos del ecosistema de Python que se describen a continuación.

### 8.1 Lenguaje de Programación: Python 3

Python es el lenguaje principal utilizado. Se eligió por su sintaxis clara, su curva de aprendizaje accesible y por ser uno de los lenguajes más utilizados en el ámbito educativo y profesional a nivel mundial.

- **Versión utilizada:** Python 3.x (compatible con Python 3.6 o superior).
- **Paradigma:** Programación imperativa y estructurada con funciones.
- **Ejecución:** Interpretado directamente desde la terminal o consola del sistema operativo.

### 8.2 Modulo random (Librería Estándar)

El único modulo externo utilizado es random, parte de la librería estándar de Python. No requiere instalación adicional.

| Funcion                         | Uso en el programa                                | Descripcion                                               |
|---------------------------------|---------------------------------------------------|-----------------------------------------------------------|
| <code>random.choice(seq)</code> | <code>random.choice(categoria['palabras'])</code> | Selecciona un elemento aleatorio de la lista de palabras. |

### 8.3 Estructuras de Datos Utilizadas

El programa hace uso de cuatro estructuras de datos nativas de Python, cada una elegida por sus características específicas:

| Estructura                      | Variable                                                          | Por qué se usa                                                                                                                                  |
|---------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>dict</code> (diccionario) | <code>CATEGORIAS</code>                                           | Permite asociar una clave numérica a un nombre y lista de palabras. Acceso directo por clave en $O(1)$ .                                        |
| <code>list</code> (lista)       | <code>HORCA</code> , <code>palabras</code>                        | Guarda elementos en orden. HORCA usa el índice del número de errores para acceder al dibujo correcto.                                           |
| <code>set</code> (conjunto)     | <code>letras_adinadas</code> ,<br><code>letras_incorrectas</code> | Garantiza unicidad (sin duplicados) y búsqueda rápida con el operador 'in' en $O(1)$ .                                                          |
| <code>str</code> (cadena)       | <code>palabra</code> , <code>nombre</code> , <code>letra</code>   | Tipo de dato para texto. Se usan métodos como <code>.lower()</code> , <code>.strip()</code> , <code>.isalpha()</code> e <code>.upper()</code> . |

## 8.4 Conceptos de Programación Aplicados

El programa integra de forma práctica los siguientes conceptos fundamentales de la programación:

| Concepto              | Como se aplica en el programa                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Funciones (def)       | El código se divide en 3 funciones: <code>mostrar_menu()</code> , <code>elegir_categoria()</code> y <code>ahorcado()</code> . Cada una tiene una responsabilidad única (principio SRP). |
| Parámetros            | La función <code>ahorcado(nombre)</code> recibe el nombre del jugador y lo utiliza en los mensajes personalizados de resultado.                                                         |
| Valor de retorno      | <code>elegir_categoria()</code> retorna el diccionario completo de la categoría elegida para que <code>ahorcado()</code> pueda usarlo.                                                  |
| Bucles while          | Se usan tres bucles <code>while</code> : para validar el nombre, para el menú de categorías, y para el juego principal.                                                                 |
| break y continue      | <code>break</code> finaliza el bucle del juego al ganar o perder. <code>continue</code> salta al inicio del bucle sin procesar el turno cuando la entrada es inválida.                  |
| Condicionales if/else | Controlan la lógica de validación, la clasificación de letras y la detección de victoria o derrota.                                                                                     |
| Comprensión de listas | La expresión <code>'c if c in letras_adivinadas else _' for c in palabra</code> genera la representación visual de la palabra en cada turno.                                            |
| f-strings             | Permiten insertar variables directamente en cadenas de texto: <code>f'Hola, {nombre}!'</code> es más legible que concatenación.                                                         |
| Recursión             | La función <code>ahorcado()</code> se llama a sí misma para iniciar una nueva partida, pasando el mismo nombre del jugador.                                                             |
| Módulos               | <code>import random</code> incorpora funcionalidad externa. El bloque <code>__name__ == '__main__'</code> controla cuando se ejecuta el código.                                         |

## 8.5 Entorno de Ejecución

El programa no requiere instalación de librerías adicionales. Para ejecutarlo, basta con tener Python 3 instalado y correr el siguiente comando en la terminal:

Terminal / Consola

```
python ahorcado.py
```

> El programa es multiplataforma: funciona en Windows (cmd o PowerShell), macOS (Terminal) y Linux (Bash) sin modificaciones. Solo se requiere Python 3.6 o superior.

## 9. Relación con el Impacto Tecnológico en la Sociedad

El desarrollo y uso de programas como el Juego del Ahorcado en Python trasciende el ámbito del entretenimiento. Representa un punto de encuentro entre la educación, la tecnología y el desarrollo social, con implicaciones concretas en múltiples dimensiones de la vida moderna.

### 9.1 Tecnología y Educación

El Juego del Ahorcado digital es un ejemplo paradigmático del uso de la tecnología como herramienta pedagógica. En el contexto del BGU ecuatoriano, este tipo de proyectos tiene un impacto directo en el proceso de enseñanza-aprendizaje.

- **Aprendizaje activo:** Los estudiantes aprenden conceptos de programación (funciones, bucles, estructuras de datos) al construir o analizar un programa con un propósito concreto y motivador.
- **Motivación intrínseca:** El formato de juego genera interés genuino en el aprendizaje. Los estudiantes no perciben el ejercicio como una tarea, sino como una actividad de disfrute con valor educativo.
- **Interdisciplinariedad:** El juego trabaja vocabulario (categorías de animales, frutas, países), pensamiento lógico (estrategia para adivinar letras) e informática simultáneamente.
- **Personalización del aprendizaje:** Las categorías de palabras pueden adaptarse a cualquier materia escolar: historia, ciencias, matemáticas, idiomas.

i Según la UNESCO, la integración de la programación en la educación básica y media es clave para preparar a las nuevas generaciones para una economía digital. Proyectos como este contribuyen directamente a ese objetivo.

### 9.2 Alfabetización Digital y Pensamiento Computacional

Programar el Ahorcado no es solo escribir código: es desarrollar una forma de pensar que tiene valor mas allá de la informática.

- **Descomposición:** El problema del juego se divide en subproblemas: registrar al jugador, elegir categoría, validar entradas, detectar fin de partida. Esta habilidad se transfiere a cualquier ambito profesional.
- **Abstracción:** El programa representa conceptos del mundo real (una palabra, una letra, un intento fallido) mediante estructuras de datos. Aprender a abstraer es una habilidad cognitiva de alto nivel.
- **Reconocimiento de patrones:** Identificar que las validaciones de entrada siguen siempre el mismo patrón (verificar, mostrar error, repetir) desarrolla la capacidad de reutilizar soluciones.
- **Algoritmos:** La lógica del juego es un algoritmo con entradas, procesos, condiciones y salidas. Comprender esto es la base del pensamiento computacional.

### 9.3 Accesibilidad e Inclusión Digital

Una de las fortalezas del programa es su bajo nivel de requerimientos técnicos, lo que lo hace accesible en contextos con recursos limitados.

- **Sin conexión a internet:** El juego funciona completamente sin conexión, siendo ideal para zonas rurales o establecimientos con conectividad limitada.

- **Hardware mínimo:** Se ejecuta en cualquier computadora con Python instalado, incluyendo equipos antiguos de baja capacidad.
- **Sin costo:** Python es de código abierto y gratuito. El programa no requiere licencias de software ni plataformas de pago.
- **Multiplataforma:** Funciona idénticamente en Windows, macOS y Linux, sin modificaciones en el código.

## 9.4 Impacto en el Desarrollo de Habilidades del Siglo XXI

El Foro Económico Mundial identifica la programación y el pensamiento crítico como habilidades esenciales para el trabajo del futuro. Este proyecto contribuye directamente al desarrollo de competencias clave:

| Habilidad               | Como la desarrolla el proyecto                                                                      |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Pensamiento crítico     | El estudiante debe analizar la lógica del programa, detectar errores y proponer mejoras.            |
| Resolución de problemas | Cada error de compilación o ejecución es un problema que debe resolverse de forma autónoma.         |
| Creatividad             | El programa puede extenderse: agregar categorías, niveles de dificultad, puntuación o temporizador. |
| Colaboración            | El proyecto puede desarrollarse en equipos, distribuyendo funciones entre los integrantes.          |
| Comunicación            | Documentar el código y presentar el proyecto desarrolla habilidades de comunicación técnica.        |
| Autonomía digital       | Aprender a crear software propio reduce la dependencia de aplicaciones comerciales.                 |

## 9.5 La Gamificación como Estrategia Pedagógica

La gamificación —el uso de elementos de juego en contextos no lúdicos— ha demostrado ser una estrategia eficaz para mejorar la motivación y la retención del aprendizaje. El Ahorcado digital es un ejemplo concreto de esta tendencia:

- Genera retroalimentación inmediata: el jugador sabe al instante si su letra es correcta o no.
- Crea tensión narrativa: los 6 intentos limitados generan una experiencia emocional que refuerza la memoria.
- Permite repetición sin frustración: el jugador puede intentar nuevamente de forma inmediata.
- Adapta la dificultad según la categoría elegida: nombres de personas puede ser más sencillo que programación.

i Estudios de la Universidad de Stanford demuestran que los estudiantes retienen hasta un 75% más de información cuando el aprendizaje se presenta en formato de juego, en comparación con métodos pasivos como la lectura.

## 9.6 Proyección y Escalabilidad del Proyecto

El programa desarrollado, aunque funcional en su estado actual, tiene un alto potencial de crecimiento y aplicación en contextos más amplios:

- **Versión gráfica:** Puede migrarse a una interfaz gráfica con tinter (incluido en Python) o pygame para una experiencia visual más rica.
- **Versión web:** Con Flask o Django (frameworks de Python) podría convertirse en una aplicación web accesible desde el navegador.
- **Multijugador:** Agregando sockets de red, podría jugarse entre estudiantes de diferentes computadoras en la misma red local.
- **Base de datos:** Integrando SQLite (incluido en Python) se podría guardar el historial de partidas y crear una tabla de puntuaciones.
- **Inteligencia Artificial:** Se podría entrenar un modelo simple de IA que adivine palabras autónomamente, convirtiendo el juego en una herramienta de estudio de machine learning.

## 9.7 Responsabilidad Ética en el Desarrollo de Software

Todo desarrollo tecnológico conlleva responsabilidades éticas. El programa del Ahorcado, aunque simple, incorpora buenas prácticas que deben fomentarse desde la formación inicial:

- **Inclusión lingüística:** Las palabras están en español, reflejando el contexto cultural del estudiante ecuatoriano.
- **Contenido apropiado:** Las categorías y palabras son 100% adecuadas para el entorno escolar.
- **Experiencia de usuario:** El programa proporciona mensajes claros, errores informativos y retroalimentación constante, respetando la experiencia del usuario.
- **Código limpio y documentado:** El uso de nombres descriptivos para variables y funciones facilita que otros puedan leer, entender y mejorar el código.

> El software responsable no solo funciona correctamente; también es comprensible, accesible, justo y respetuoso con sus usuarios. Estos valores deben inculcarse desde los primeros pasos en la programación.

## 9.8 Conclusión

El Juego del Ahorcado en Python es mucho más que un programa de consola: es una demostración práctica de cómo la tecnología puede transformar la educación, democratizar el acceso al conocimiento y desarrollar competencias fundamentales para el siglo XXI.

En el contexto de la Unidad Educativa Fiscal Primicias de la Cultura de Quito, proyectos como este representan un puente concreto entre el currículo académico y las demandas del mundo digital contemporáneo. Cada estudiante que comprende, crea o mejora este programa da un paso hacia la alfabetización computacional que el Ecuador y el mundo necesitan.

La simplicidad del lenguaje Python, combinada con la familiaridad del juego del ahorcado, crea el entorno perfecto para que estudiantes de todos los niveles descubran que la programación no es un campo reservado para expertos, sino una habilidad creativa, accesible y profundamente humana.